Training objective $\mathcal{L}_{\text{VLB}}$. Inherited from Diffuseq [14], the training objective is to recover the original $\mathbf{z}_0$ by denoising $\mathbf{z}_t$ as in Eq. (8). The learning process as Eq. (9) is modeled by Diffuseq: $\mathbf{z}_\theta(\mathbf{z}_t, t)$, where the $\mu_\theta(\cdot)$ and $\sigma_\theta(\cdot)$ serve as the parameterization of the predicted mean and standard deviation of $q(\mathbf{z}_{t-1}|\mathbf{z}_t)$ in the $forward\ noising$ process respectively. The input $\mathbf{x}_t$ serves as the condition during the $reverse\ denoising$ process as the partial noising is adopted in the $forward\ noising$.

Typically, a transformer architecture is adopted to model $\mathbf{z}_\theta$, which is capable of modeling the semantic relation between $\mathbf{x}_t$ and $\mathbf{y}_t$ instinctively. The variational lower bound ($\mathcal{L}_{\text{VLB}}$) is computed as follows:

$$\mathcal{L}_{\text{VLB}}(\mathbf{w}^z) = \mathbb{E}_{q(\mathbf{z}_0|\mathbf{w}^z)}\left[\underbrace{\log \frac{q(\mathbf{z}_T|\mathbf{w}^z)}{p_\theta(\mathbf{z}_T)}}_{\text{Prior loss}} + \underbrace{\mathcal{L}_{\text{VLB}}(\mathbf{z}_0)}_{\text{Diffusion loss}} \underbrace{- \log p_\theta(\mathbf{w}^z|\mathbf{z}_0)}_{\text{Rounding loss}}\right], \tag{10}$$

where the diffusion loss is the same as the continuous diffusion loss in DDPM [21], which is given by:

$$\mathcal{L}_{\text{VLB}}(\mathbf{z}_0) = \mathbb{E}_{q(\mathbf{z}_{1:T}|\mathbf{z}_0)}\left[\underbrace{\log \frac{q(\mathbf{z}_T|\mathbf{z}_0)}{p_\theta(\mathbf{z}_T)}}_{\mathcal{L}_T} + \underbrace{\sum_{t=2}^{T} \log \frac{q(\mathbf{z}_{t-1}|\mathbf{z}_0, \mathbf{z}_t)}{p_\theta(\mathbf{z}_{t-1}|\mathbf{z}_t)}}_{\mathcal{L}_{T-1}+\cdots+\mathcal{L}_1} \underbrace{-\log p_\theta(\mathbf{z}_0|\mathbf{z}_1)}_{\mathcal{L}_0}\right]. \tag{11}$$

Here the prior loss and $\mathcal{L}_T$ is considered as a constant when the noising schedule q is fixed and $p_\theta(\mathbf{z}_T) = \mathcal{N}(0, \mathbf{I})$.

After reweighting each term (i.e., treating all the loss terms across time-steps equally) as in Ho et al. [21] and using the Monte Carlo optimizer, the training objective can be further simplified as:

$$\begin{aligned}
\min_\theta \ \mathcal{L}_{\text{VLB}}(\mathbf{w}^z) &\rightarrow \min_\theta \mathbb{E}_{q(\mathbf{z}_{0:T}|\mathbf{w}^z)}\left[\sum_{t=2}^{T} ||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t, t)||^2 + ||\textsc{Emb}(\mathbf{w}^z) - \mathbf{z}_\theta(\mathbf{z}_1, 1)||^2 - \log p_\theta(\mathbf{w}^z|\mathbf{z}_0)\right] \\
&\rightarrow \min_\theta \left[\sum_{t=2}^{T} ||\mathbf{y}_0 - \tilde{\mathbf{z}}_\theta(\mathbf{z}_t, t)||^2 + ||\textsc{Emb}(\mathbf{w}^y) - \tilde{\mathbf{z}}_\theta(\mathbf{z}_1, 1)||^2 + \mathcal{R}(||\mathbf{y}_0||^2)\right] \\
&\rightarrow \min_\theta \left[\sum_{t=1}^{T} ||\mathbf{y}_0 - \tilde{\mathbf{z}}_\theta(\mathbf{z}_t, t)||^2 + \mathcal{R}(||\mathbf{y}_0||^2)\right],
\end{aligned} \tag{12}$$

where $\tilde{\mathbf{z}}_\theta(\mathbf{z}_t, t)$ is used to denote the fractions of recovered $\mathbf{z}_0$ corresponding to $\mathbf{y}_0$. $\mathcal{R}(||\mathbf{y}_0||^2))$ is the regularization term which regularizes the embedding learning. The embedding function is shared between source and target sequences, contributing to the joint training process of two different feature spaces.

A.2 Pre-trained Plaid

The Plaid model [18] mostly adopts the variational diffusion model (VDM) framework [28] and we illustrate its forward, reverse, and loss calculations in this section. When fine-tuning Plaid 1B, we use the VDM formulation and apply the same sequence-to-sequence modification as in DiffuSeq. This involves imposing partial noise on $\mathbf{z}_t$ and keeping the source condition sentence anchored as un-noised.

Forward diffusion process $q(\mathbf{z}_t|\mathbf{z}_0)$ and $q(\mathbf{z}_t|\mathbf{z}_s)$. The distribution of latent $\mathbf{z}_t$ conditioned on $\mathbf{z}_0$ is given by:

$$q(\mathbf{z}_t|\mathbf{z}_0) = \mathcal{N}(\alpha_t \mathbf{z}_0, \sigma_t^2 \mathbf{I}). \tag{13}$$

After reparameterization, we have $\mathbf{z}_0 = (\mathbf{z}_s - \epsilon_1 \sigma_s)/\alpha_s$ and $\mathbf{z}_t = (\alpha_t/\alpha_s)\mathbf{z}_s - (\alpha_t \sigma_s/\alpha_s)\epsilon_1 + \sigma_t \epsilon_2$, where $\epsilon_1 \sim \mathcal{N}(0, \mathbf{I})$ and $\epsilon_2 \sim \mathcal{N}(0, \mathbf{I})$. Then after merging two uniform Gaussians, the distribution of $\mathbf{z}_t$ given $\mathbf{z}_s$, for any $0 \leq s < t \leq 1$, is given by:

$$q(\mathbf{z}_t|\mathbf{z}_s) = \mathcal{N}\left(\alpha_{t|s}\mathbf{z}_s, \sigma_{t|s}^2 \mathbf{I}\right), \tag{14}$$

where $\alpha_{t|s} = \alpha_t/\alpha_s$ and $\sigma^2_{t|s} = \sigma^2_t - \alpha^2_{t|s}\sigma^2_s$. The variance-preserving special case gives $\alpha_t = \sqrt{1-\sigma^2_t}$. In VDM, the noise schedule α_t and σ^2_t, which specify how much noise to add at each time in the diffusion process, are parameterized as a scalar-to-scalar neural network $\boldsymbol{\eta}$ that satisfies $\sigma^2_t = \text{sigmoid}\left(\gamma_{\boldsymbol{\eta}}(t)\right)$ and $\alpha^2_t = \text{sigmoid}\left(-\gamma_{\boldsymbol{\eta}}(t)\right)$. This is different from previous practices that use a predefined function, e.g., DDPM [21] set the forward process variances to constants increasing linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$.

Posterior $q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)$. The joint distribution of latent variables $(\mathbf{z}_s, \mathbf{z}_t, \mathbf{z}_u)$ at any subsequent timesteps $0 \le s < t < u \le 1$ is Markov: $q(\mathbf{z}_u|\mathbf{z}_t, \mathbf{z}_s) = q(\mathbf{z}_u|\mathbf{z}_t)$. Given the distributions above, we can verify through the Bayes rule that $q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)$, for any $0 \le s < t \le 1$, is also Gaussian given by:

$$q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0) = \mathcal{N}(\mu_Q(\mathbf{z}_t, \mathbf{z}_0; s, t), \sigma^2_Q(s,t)\mathbf{I}) \tag{15}$$

$$\text{where } \sigma^2_Q(s,t) = \sigma^2_{t|s}\sigma^2_s/\sigma^2_t \tag{16}$$

$$\text{and } \mu_Q(\mathbf{z}_t, \mathbf{z}_0; s, t) = \frac{\alpha_{t|s}\sigma^2_s}{\sigma^2_t}\mathbf{z}_t + \frac{\alpha_s\sigma^2_{t|s}}{\sigma^2_t}\mathbf{z}_0. \tag{17}$$

Backward generative process $p_\theta(\mathbf{z}_s|\mathbf{z}_t)$. In VDM, the reverse process or the generative process is also defined as a Gaussian that satisfies $p_\theta(\mathbf{z}_s|\mathbf{z}_t) = q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0 = \mathbf{z}_\theta(\mathbf{z}_t; t))$, i.e. the same as $q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)$, but with the original data $\mathbf{z}_0$ replaced by the output of the denoising model $\mathbf{z}_\theta(\mathbf{z}_t; t)$. Therefore, based on Eq. (17), the mean of $p_\theta(\mathbf{z}_s|\mathbf{z}_t)$ is given by:

$$\mu_\theta(\mathbf{z}_t; s, t) = \frac{\alpha_{t|s}\sigma^2_s}{\sigma^2_t}\mathbf{z}_t + \frac{\alpha_s\sigma^2_{t|s}}{\sigma^2_t}\mathbf{z}_\theta(\mathbf{z}_t; t), \tag{18}$$

and the variance is the same as Eq. (16).

Continuous diffusion loss term $\mathcal{L}_{\text{VLB}}(\mathbf{z}_0)$. The prior loss and rounding loss in Eq. (10) can be (stochastically and differentiably) estimated using standard techniques. We now derive an estimator for the diffusion loss in VDM. Different from Eq.(12) which simplifies the loss term by reweighting, VDM adopts the standard loss formulation. We begin with the derivations of diffusion loss for discrete-time diffusion with $t \in \{1, \ldots, T\}$, which is given by:

$$\mathcal{L}_{\text{VLB}}(\mathbf{z}_0) = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_t|\mathbf{z}_0)} D_{KL}[q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)||p(\mathbf{z}_s|\mathbf{z}_t)], \tag{19}$$

and we derive the expression of $D_{KL}(q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)||p_\theta(\mathbf{z}_s|\mathbf{z}_t))$ as follows:

$$\begin{aligned}
D_{KL}(q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)||p_\theta(\mathbf{z}_s|\mathbf{z}_t)) &= \frac{1}{2\sigma^2_Q(s,t)}||\mu_Q - \mu_\theta||^2_2 && (20)\\
&= \frac{\sigma^2_t}{2\sigma^2_{t|s}\sigma^2_s}\frac{\alpha^2_s\sigma^4_{t|s}}{\sigma^4_t}||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2 && (21)\\
&= \frac{1}{2\sigma^2_s}\frac{\alpha^2_s\sigma^2_{t|s}}{\sigma^2_t}||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2 && (22)\\
&= \frac{1}{2\sigma^2_s}\frac{\alpha^2_s(\sigma^2_t - \alpha^2_{t|s}\sigma^2_s)}{\sigma^2_t}||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2 && (23)\\
&= \frac{1}{2}\frac{\alpha^2_s\sigma^2_t/\sigma^2_s - \alpha^2_t}{\sigma^2_t}||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2 && (24)\\
&= \frac{1}{2}\left(\frac{\alpha^2_s}{\sigma^2_s} - \frac{\alpha^2_t}{\sigma^2_t}\right)||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2 && (25)\\
&= \frac{1}{2}\left(\text{SNR}(s) - \text{SNR}(t)\right)||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||^2_2, && (26)
\end{aligned}$$

where $\text{SNR}(t) = \alpha^2_t/\sigma^2_t$ and its physical meaning is signal-to-noise ratio.

After reparameterization of $\mathbf{z}_t$, the diffusion loss function becomes:

$$\mathcal{L}_{\text{VLB}}(\mathbf{z}_0) = \sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{z}_t|\mathbf{z}_0)}[D_{KL}(q(\mathbf{z}_s|\mathbf{z}_t, \mathbf{z}_0)||p_\theta(\mathbf{z}_s|\mathbf{z}_t))] \tag{27}$$

$$= \frac{1}{2}\mathbb{E}_{\epsilon\sim\mathcal{N}(0,\mathbf{I})}[\sum_{t=1}^{T} (\text{SNR}(s) - \text{SNR}(t)) \, ||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||_2^2]. \tag{28}$$

In practice, we follow Plaid to use the continuous-time diffusion formulation, where $t \in [0, 1]$, and we can express $\mathcal{L}$ as a function of τ with $\tau \to 0$:

$$\mathcal{L}_{\text{VLB}}(\mathbf{z}_0) = \frac{1}{2}\mathbb{E}_{\epsilon\sim\mathcal{N}(0,\mathbf{I})} \int_0^1 \left[\frac{\text{SNR}(t-\tau) - \text{SNR}(t)}{\tau} ||\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)||_2^2\right] dt, \tag{29}$$

and let $\text{SNR}'(t)$ denote the derivative of the SNR function, this then gives:

$$\mathcal{L}_{\text{VLB}}(\mathbf{z}_0) = -\frac{1}{2}\mathbb{E}_{\epsilon\sim\mathcal{N}(0,\mathbf{I})} \int_0^1 \text{SNR}'(t) \, \|\mathbf{z}_0 - \mathbf{z}_\theta(\mathbf{z}_t; t)\|_2^2 \, dt. \tag{30}$$

A.3 Pre-trained SEDD

SEDD [37] is a discrete diffusion language model built based on discrete score matching [40], which generalizes score matching [45, 46] to the discrete data. We now denote x as a categorical random variable and the following derivation can be extended to a sequence of variable $\mathbf{x}$ as well.

Concrete score. Instead of directly modeling $p_\theta(x)$ to approximate original data distribution $q(x)$, the core idea of discrete score matching is to learn a quantity known as the *concrete score* [40] through a neural network:

$$s_\theta(x)_y = \frac{p_\theta(y)}{p_\theta(x)} = \frac{e^{f_\theta(y)}/Z}{e^{f_\theta(x)}/Z} = \frac{e^{f_\theta(y)}}{e^{f_\theta(x)}}, \tag{31}$$

which eliminates normalizing constant Z as in the energy-based model. In particular, this quantity is the categorical equivalent of the famous score function $\nabla_x \log p$ in continuous space. Regarding the choice of y, if we model the ratio for every possible y, we would have V items given V as the dimension of x, and N^V items for $\mathbf{x}$ given N as the sequence length of $\mathbf{x}$, which is computationally intractable. So we sparsify and only model "relevant" ratios based on whether y is "close" to x. These relevant positions will be denoted as $y \sim x$, e.g., all sentences y that differ from x with Hamming distance 1.

Training objective. Lou et al. [37] define a learning objective named *score entropy* to learn the neural network, which is given by:

$$\mathbb{E}_{x\sim q}\left[\sum_{y\sim x} s_\theta(x)_y - \frac{q(y)}{q(x)} \log s_\theta(x)_y\right]. \tag{32}$$

Taking a derivative w.r.t. s and setting it to 0, we see that this occurs when $s_\theta(x)_y = \frac{q(y)}{q(x)}$, which can be easily checked to be globally optimal as the function is convex as a function of s. To handle the unknown term $\frac{q(y)}{q(x)}$, they further propose *denoising score entropy* motivated by *denoising score matching* [45] based on $q(x_t) = \sum_{x_0} q_{t|0}(x_t|x_0)q_0(x_0)$:

$$\mathbb{E}_{x_0\sim q_0, t\sim U[0,T], x_t\sim q_{t|0}(x_t|x_0)}\left[\sum_{y\sim x_t} s_\theta(x_t, t)_y - \frac{q_{t|0}(y|x_0)}{q_{t|0}(x_t|x_0)} \log s_\theta(x_t, t)_y\right], \tag{33}$$

where $q_{t|0}(\cdot|x_0)$ is a perturbation of a base density $q(\cdot)$ by a transition kernel, and the transition ratio $\frac{q_{t|0}(y|x_0)}{q_{t|0}(x_t|x_0)}$ is known by design.